

Module: Les technologies du Middleware



Cours 3: *Middleware* à objets répartis (Object Oriented
Middleware)



- Définition et propriétés des OOM (Object oriented Middlewares)
- Notions sur les objets.
- Qu'attend t-on d'un objet distribué.
- Objets distants: Notion de référence
- Architecture d'une application basé sur MOO
- MOO: Problèmes

Propriétés des MOO (Middleware Orienté Objet)

- Prise en charge du modèle de programmation orienté objet (objets, méthodes, interfaces, encapsulation,...)

Exceptions: étaient également dans certains systèmes RPC, par exemple Mayflower

- Requête/réponse: interaction généralement synchrone comme le RPC.

- Transparence de l'emplacement, le système ORB (*Object Request Broker*) mappe les références d'objet aux emplacements

- Les services comprenant plusieurs serveurs sont plus faciles à créer avec MOO. Le système RPC recherche l'adresse du serveur dans un service de localisation.



■ Encapsulation

L'interface (méthodes + attributs) est la seule voie d'accès à l'état interne, non directement accessible

■ Classes et instances

Mécanismes de génération d'exemplaires conformes à un même modèle

■ Héritage

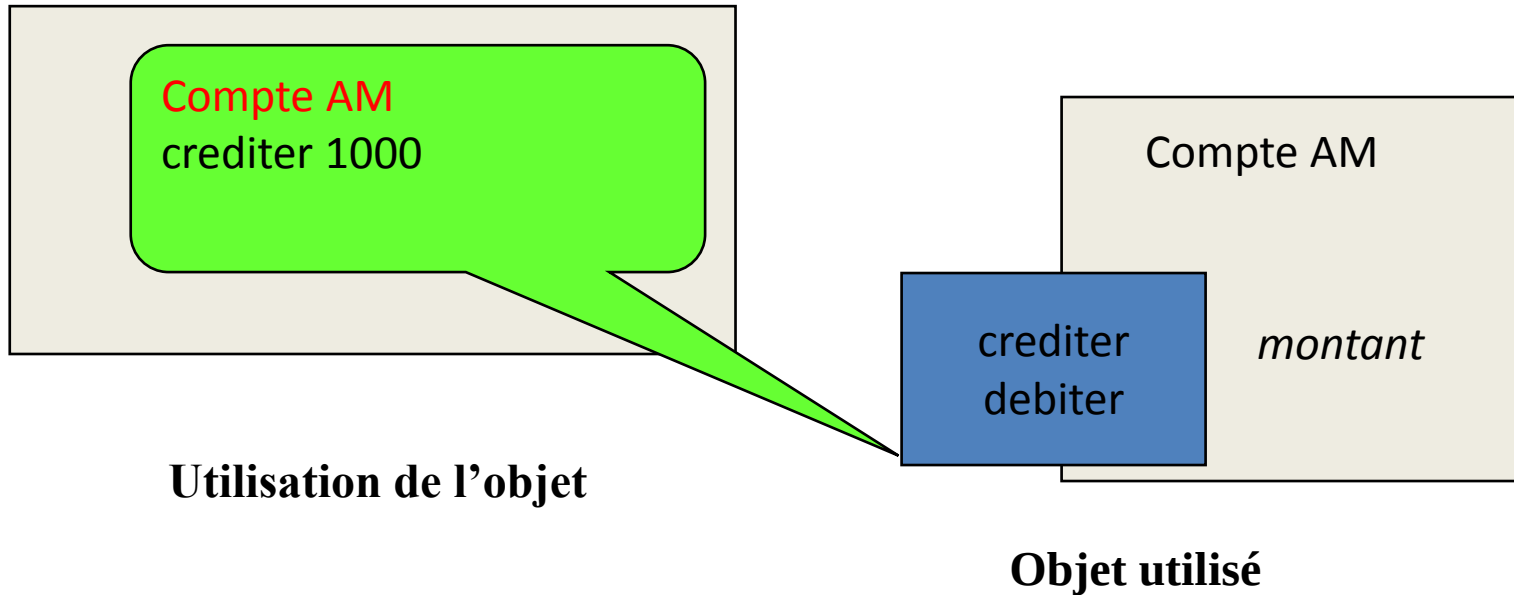
Mécanisme de spécialisation : facilite récupération et réutilisation de l'existant

Intérêt des objets pour la construction d'applications réparties (2)



■ Polymorphisme

- Mises en œuvre diverses des fonctions d'une interface
- Remplacement d'un objet par un autre si interfaces "compatibles"
- Facilite l'évolution et l'adaptation des applications



Interface = partie visible de l'objet

Implémentation = partie privée inaccessible depuis d'autres objets

Modifier l'implémentation des classes sans altérer l'utilisation

Interface = contrat entre l'objet et le monde extérieur

Appel Local: exemple (1)



```
public interface ReverseInterface {  
    String reverseString(String chaine);  
}
```

```
public class Reverse implements ReverseInterface  
{  
    public String reverseString (String ChaineOrigine){  
  
        int longueur=ChaineOrigine.length();  
        StringBuffer temp=new StringBuffer(longueur);  
        for (int i=longueur; i>0; i--) {  
            temp.append(ChaineOrigine.substring(i-1, i));  
        }  
        return temp.toString();  
    }  
}
```

Appel Local: exemple (2)



```
import ReverseInterface;

public class ReverseClient
{
    public static void main (String [] args)
    {
        Reverse rev = new Reverse();
        String result = rev.reverseString (args [0]);
        System.out.println ("L'inverse de "+args[0]+" est
"+result);
    }
}
```

```
$javac *.java
$java ReverseClient Alice
L'inverse de Alice est ecilA
$
```

Qu'attend t-on d'un objet distribué ? (1)



Un objet distribué doit pouvoir être vu comme un objet « normal ».

Soit la déclaration suivante :

ObjetDistribue monObjetDistribue;

On doit pouvoir invoquer une méthode de cet objet situé sur une autre machine de la même façon qu'un objet local :

monObjetDisribue.uneMethodeDeLOD();

Qu'attend t-on d'un objet distribué ? (2)



■ On doit pouvoir utiliser cet objet distribué sans connaître sa localisation. On utilise pour cela un service sorte d'annuaire, qui doit nous renvoyer son adresse.

```
monObjetDistribue=ServiceDeNoms.recherche('myDistributedObject');
```

■ On doit pouvoir utiliser un objet distribué comme paramètre d'une méthode locale ou distante.

```
x=monObjetLocal.uneMethodeDeLOL(monObjetDistribue);
```

```
x=monObjetDistribue.uneMethodeDeLOD(autreObjetDistribue);
```

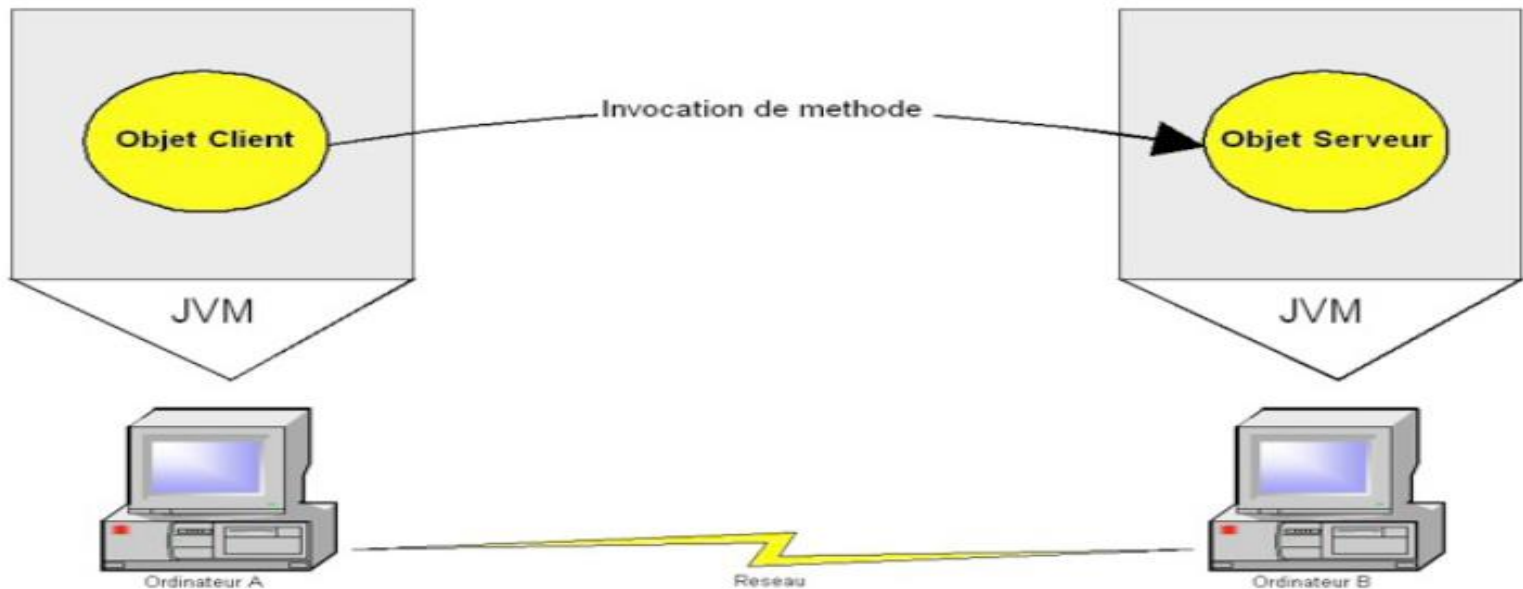
Qu'attend t-on d'un objet distribué ? (3)



On doit pouvoir récupérer le résultat d'un appel de méthode sous la forme d'un objet distribué.

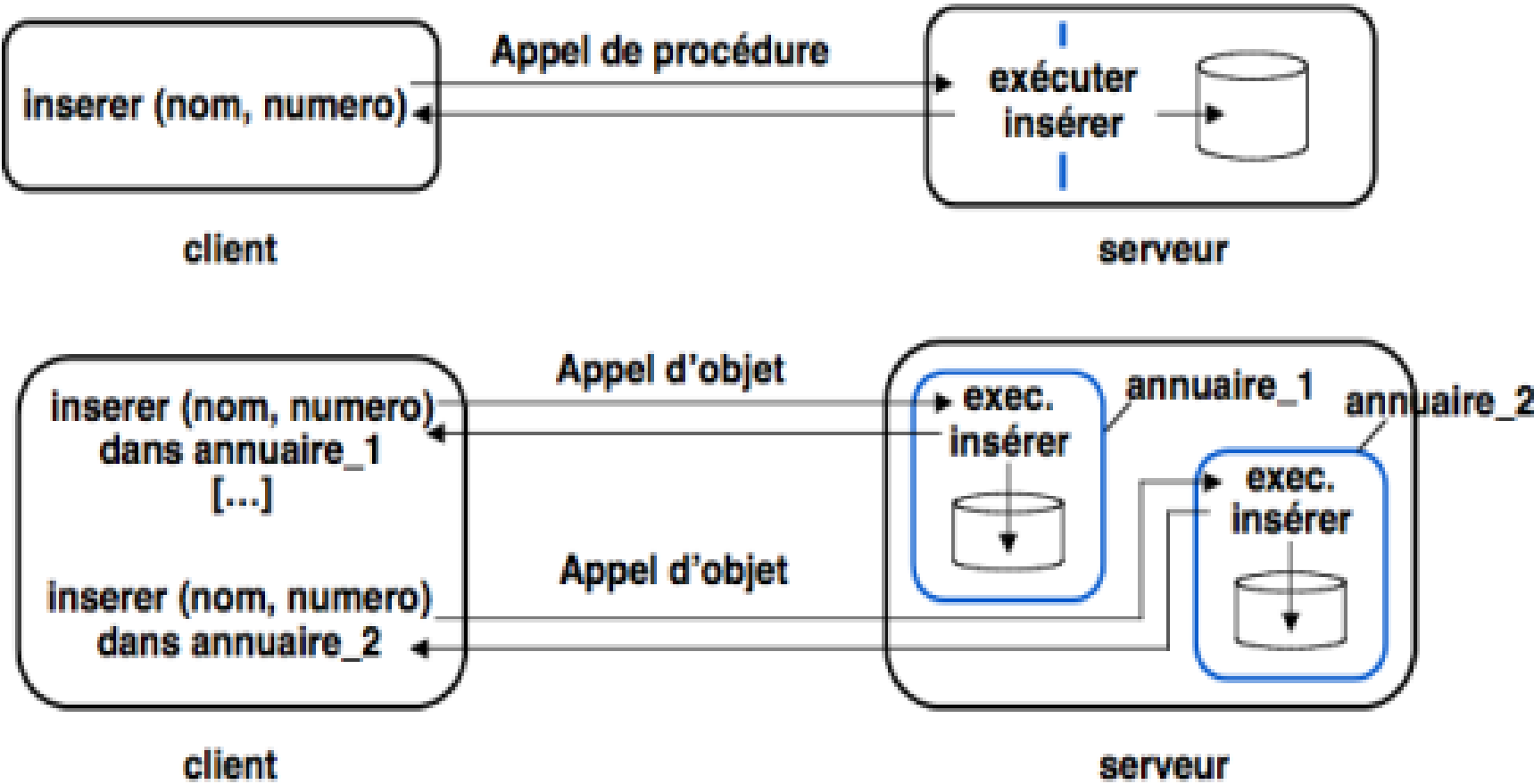
`monObjetDistribue=autreObjetDistribue.uneMethodeDeLOD();`

`monObjetDistribue=monObjetLocal.uneMethodeDeLOL();`





Appel de procédure vs appel de méthode sur un objet (1)

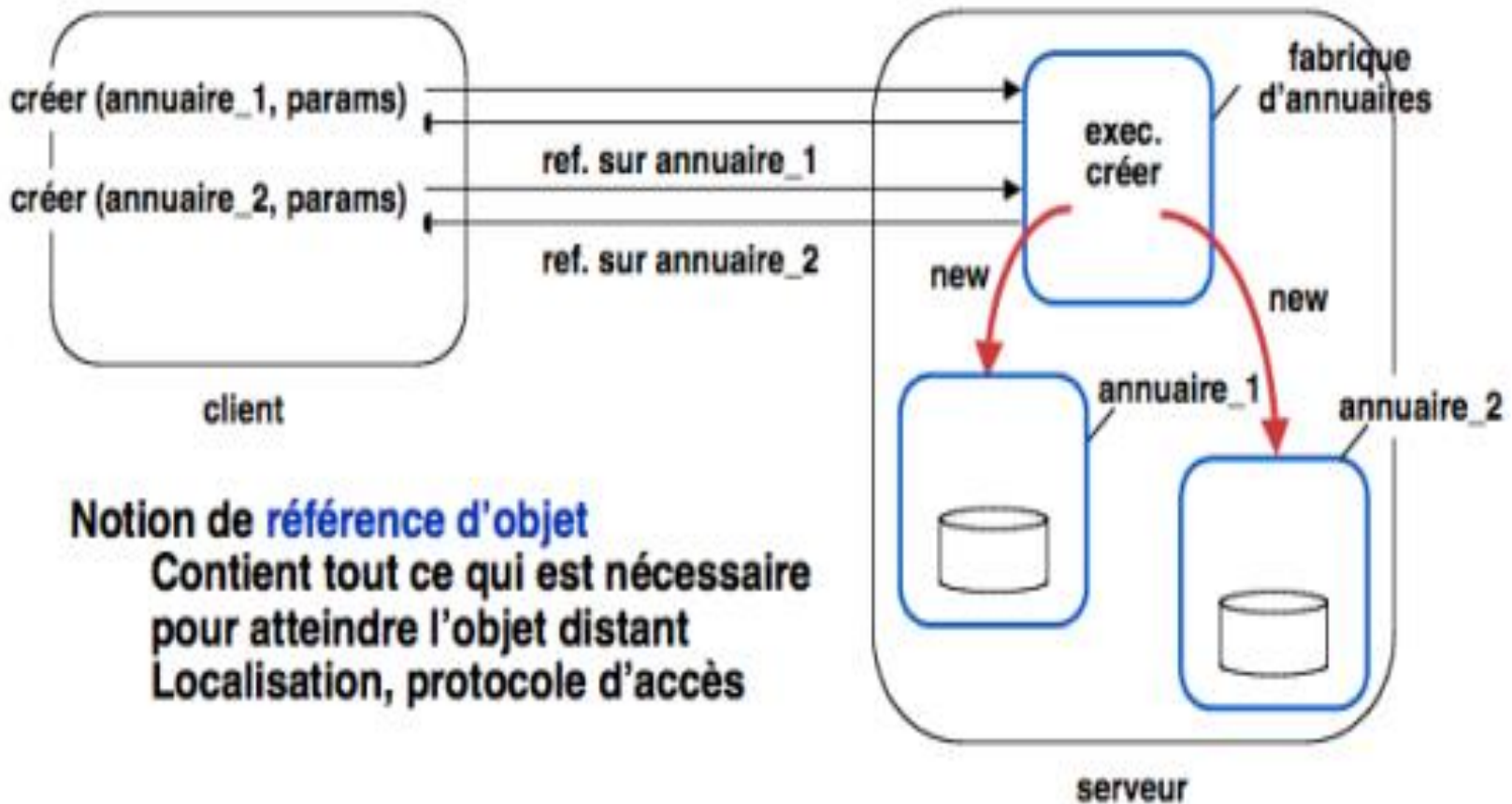


Exemple : insérer une entrée dans un annuaire



Appel de procédure vs appel de méthode sur un objet (2)

Création d'instances d'une classe d'objets





Référence : notion clé pour les objets répartis

- ❑ Moyen d'accès à un objet distant
- ❑ Son contenu ne peut pas être directement exploité
- ❑ Elle n'est utilisable que via un mécanisme d'appel à distance

Contenu d'une référence: Toutes les informations nécessaires pour atteindre physiquement l'objet

- Site de résidence (adresse IP)
- Numéro de porte sur le site (Numéro de port)
- Localisation interne au serveur
- ...



Exemples

- En Java RMI : la référence est le talon client (stub)
- En Corba : format de référence spécifié (IOR : Interoperable Object Reference)
- En DCOM : format “propriétaire”



Resultat = **ObjetL.Metho(Param)**

Résultat

Objet cible

Opérateur d'accès

Nom de la méthode

Liste des paramètres

Resultat = **ObjetD.MethoD(Param)**

Résultat (renvoyé
par l'objet distant
via le réseau)
(encapsule l'adresse et le protocole)

Objet Cible

Opérateur d'accès
(Garanti la
transparence de
localisation)

Nom de
la méthode

Liste des paramètres
(Envoyée via le réseau
à l'objet distant)



- Pour qu'on puisse invoquer les méthodes des objets distants, on a besoin de leurs références. D'où la notion de broker (ORB) qui est un annuaire contenant les références d'objets d'un ou plusieurs serveurs.
- Dans les RPC, les clients doivent connaître l'adresse du serveur qui contient l'implémentation des procédures distantes

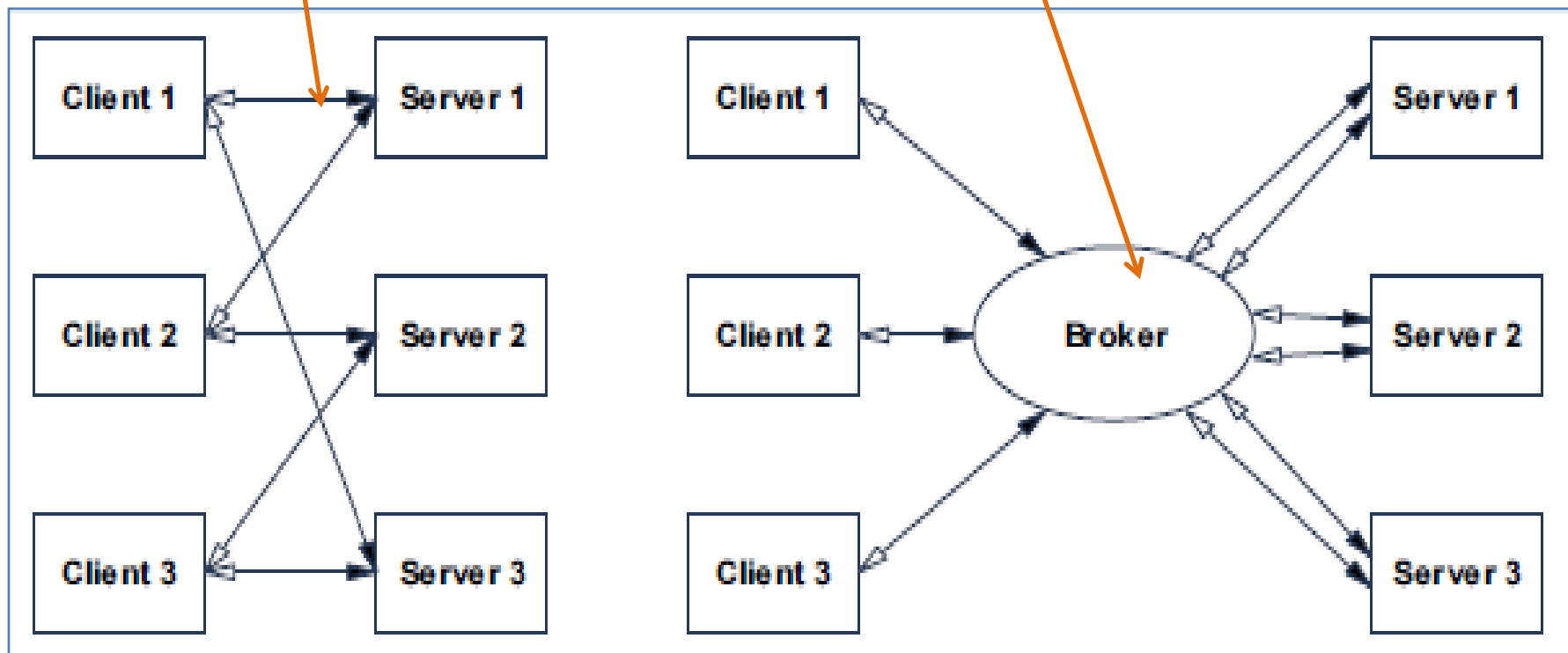
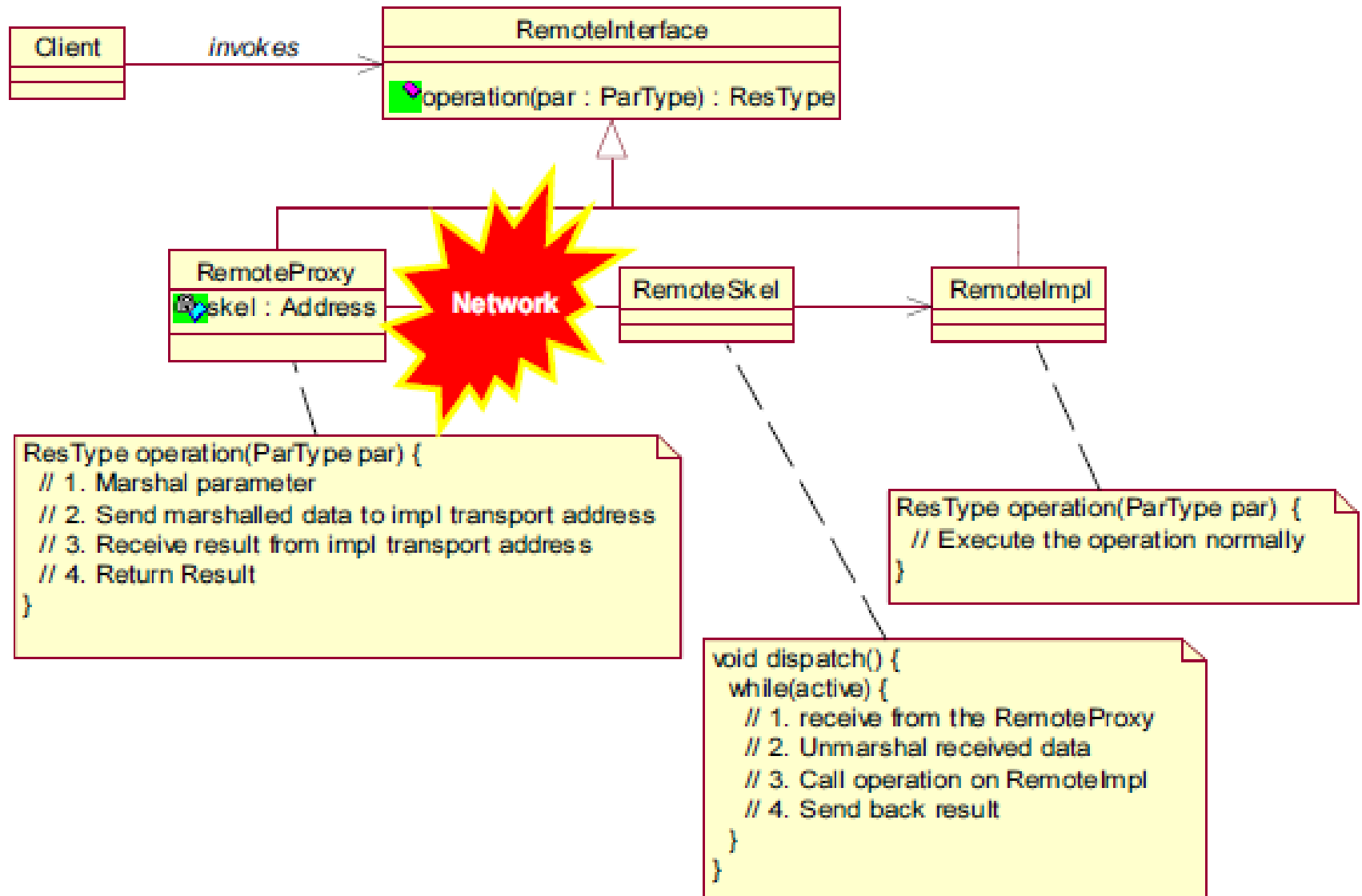


Diagramme de classe type d'un MOO





Le modèle ISO 7 couches et Middleware MOO



Application distribuées à objets répartis

Middleware MOO

Les middlewares orientés objets ou bien à objets répartis agissent au niveau des couches présentation et session.

➤ La couche session établit et maintient les connections entre les composants d'un système distribué

➤ La couche présentation gère les problèmes liés à l'hétérogénéité de données et assure le transport de données complexes.

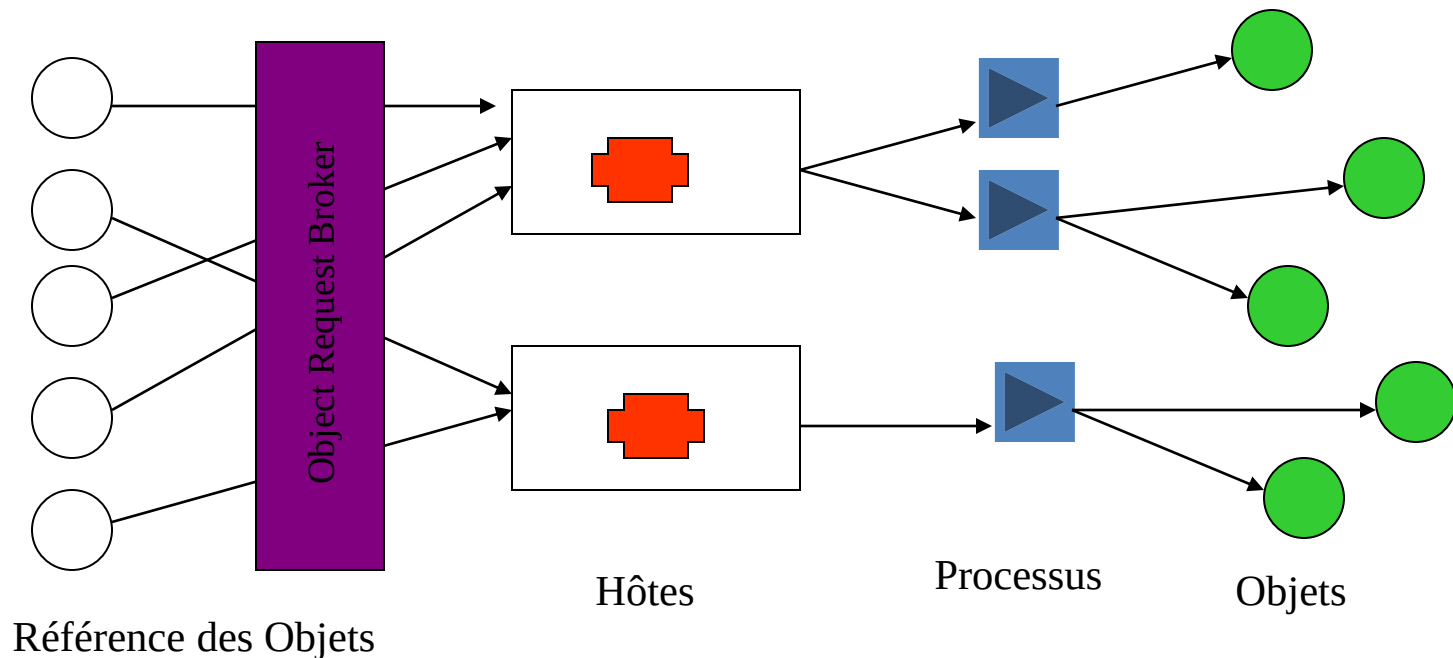


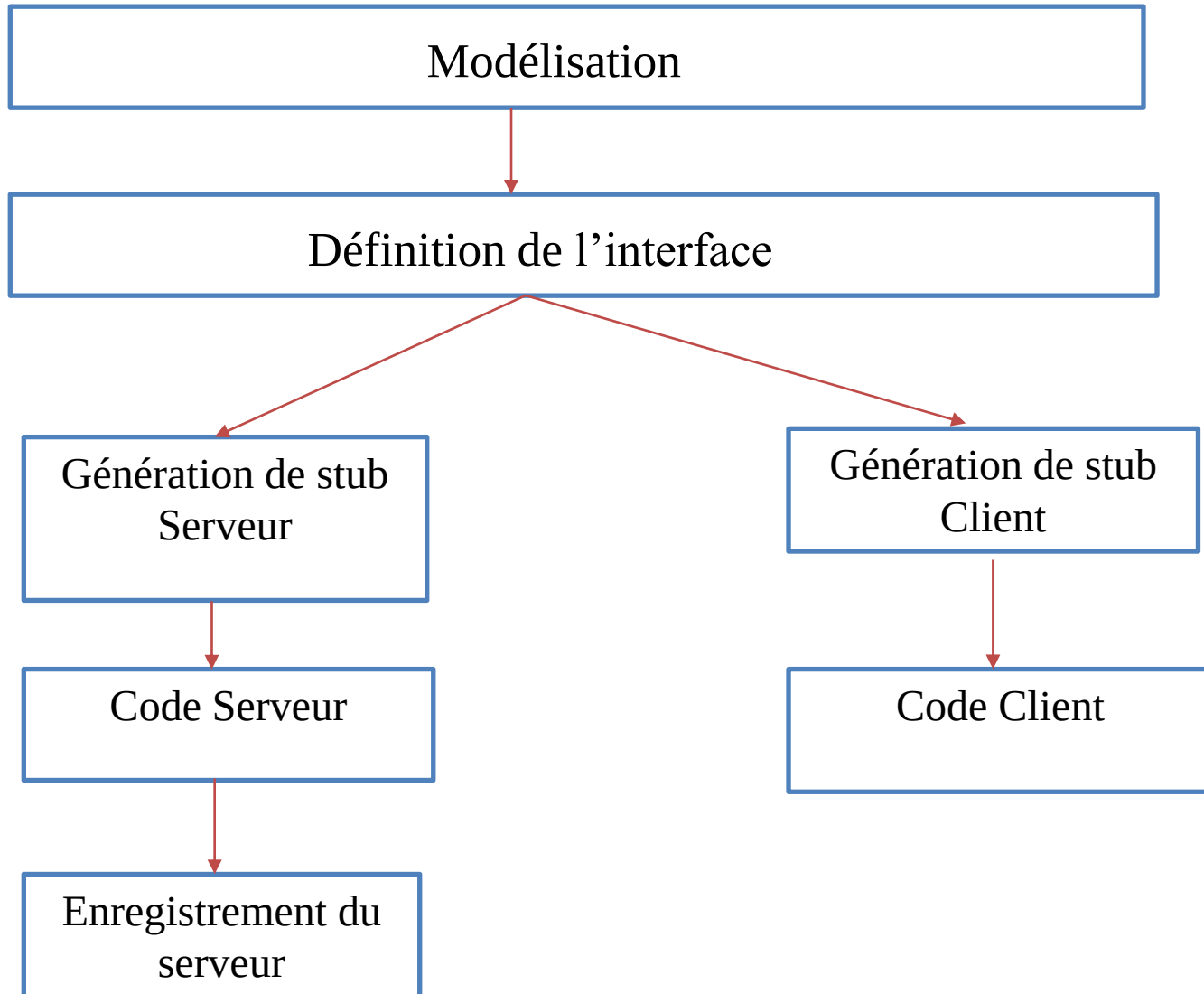
- Similaire au RPC
 - Supporte les stubs de client et de serveur.
 - Réalise le marshalling and unmarshalling
 - Résout le problème de l'hétérogénéité des données.
- Différent de RPC
 - Définit la représentation des références d'objets
 - Marshalling/unmarshalling des références d'objets



MOO: au niveau de la couche session

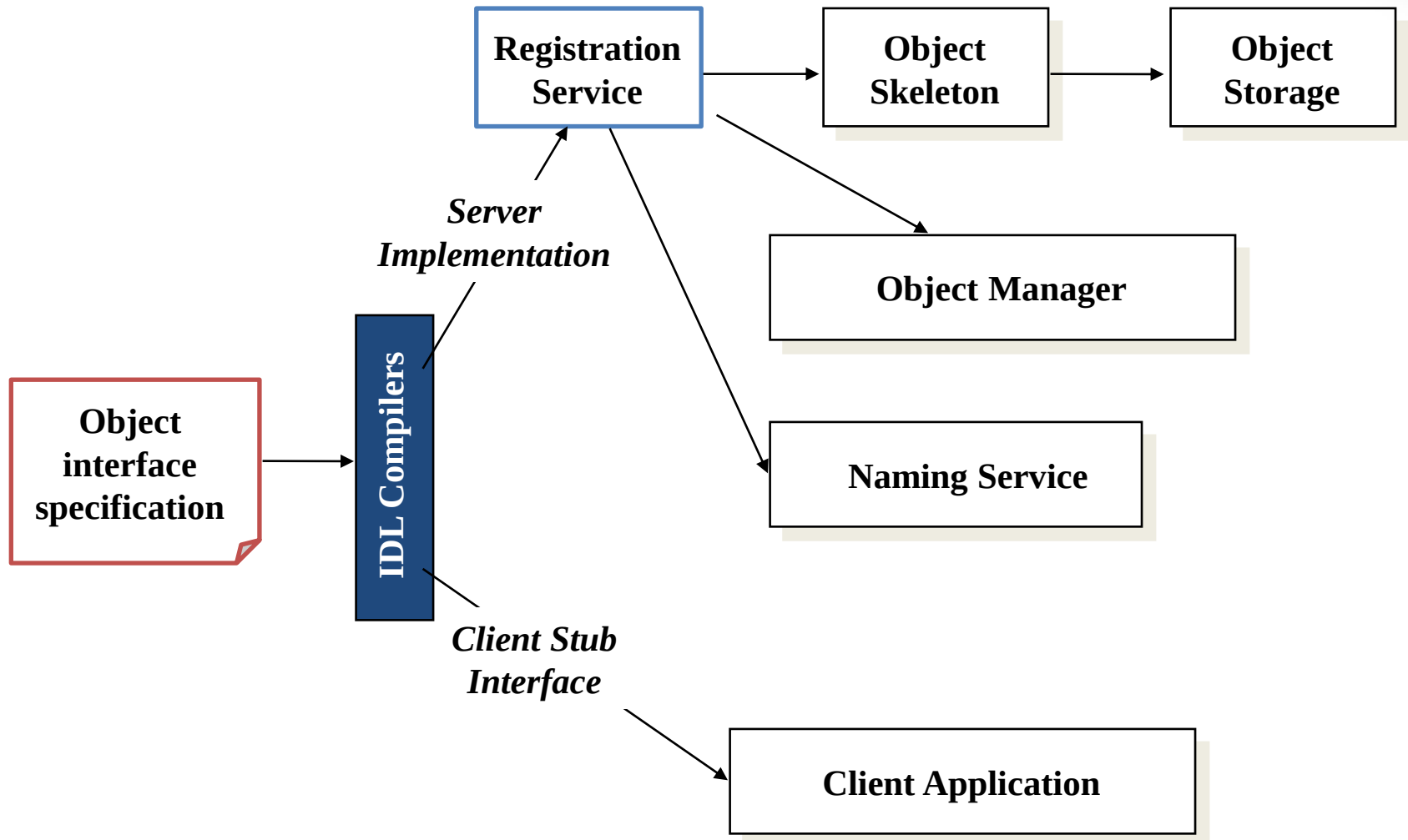
- Mappe une référence d'objet à un hôte
- Implémente les conditions d'activation dans l'adaptateur d'objets
- L'adaptateur d'objets a besoin d'être capable de démarrer le serveur, qui est enregistré dans un répertoire d'enregistrement ou un registre.
- Implémente le dispatcheur d'opérations
- Implémente la synchronisation







Architecture générale d'un système distribuée OO





- Requête/réponse: communication synchrone, Pour CORBA
Sémantique oneway ajoutée et invocation de méthode asynchrone (AMI)
- Implémentation avec un couplage fort.
- Une collection d'objets distribués parfois inutilisés ce qui produit une mémoire perdue.
- MOO plutôt statique et lourd Mauvais pour les systèmes omniprésents et les périphériques embarqués



1. Samia Bouzefrane, Cours : ***Intergiciels à Objets Répartis : Java RemoteMethod Invocation (RMI) ou les invocations de méthodes Java distantes.***
2. Cours de S. Krakowiak.
3. Andrea Omicini *after Giovanni, Rimassa*, Object-Oriented Middleware for Distributed Systems.
4. Douglas C. Schmidt, Stephen D. Huston, C++ Network Programming, Volume I Mastering Complexity with ACE and Patterns.